

lec 85

Q.1) Sum of longest path from root to leaf node.

~~int sum(Node*~~

void solve (Node* root, int sum,
int &maxSum, int len, int &max
len) {

if (root == NULL) {

~~return;~~

8

if (len > maxlen) {

maxlen = len;

maxSum = sum;

}

```

else if (len == maxlen) {
    maxSum = max(sum, maxSum);
}
return;

```

```

}

sum = sum + root->data;
solve(root->left, sum, maxSum
    maxSum, len+1, maxlen);

solve(root->right, sum, maxSum,
    len+1, maxlen);

```

```

}

int sumOfMaxPath (Node* root) {
    int len = 0;
    int maxlen = 0;
    int sum = 0;
    int maxSum = 0;

    solve (root, sum, maxSum,
        len+1, maxlen);

    return maxSum;
}

```

Q.2) Lowest Common Ancestor
in a Binary Tree.

i/p n_1, n_2 and Tree.

o/p → Lowest common node of them.

Logic : Jab dono side se Non Null ans aaye tab wo hi ans hai

```

Node* lca (Node* root, int n1,
    int n2) {

```

// base case

```

if (root == NULL) {
    return NULL;
}

```

```

if (root->data == n1 ||
    root->data == n2) {
    return root;
}

```

```

Node* leftAns = lca (root->left,
    n1, n2);

```

```

Node* rightAns = lca (root->right,
    n1, n2);

```

```

if (leftAns != NULL &&
    rightAns != NULL) {
    return root;
}

```

```

else if (leftAns != NULL &&
    rightAns == NULL) {
    return leftAns;
}

```

```

else if (leftAns == NULL &&
    rightAns != NULL) {
    return rightAns;
}
else {
    return NULL;
}

```

$T(n) = O(n)$
 $S(n) = O(h)$

Q.3) K Sum Paths.

→ Return total no. of paths whose sum is equal to K.

Path (downwards direction).

```
void solve (Node* root, int k,
            int &count, vector<int> &path) {
```

```
    if (root == NULL)
        return;
```

```
    path.push_back (root->data);
```

```
    // left
```

```
    solve (root->left, k, count,
           path)
```

```
    solve (root->right, k, count,
           path)
```

```
    int size = path.size();
```

```
    int sum = 0;
```

```
    for (int i = size-1; i >= 0; i--) {
```

```
        sum += path[i];
```

```
        if (sum == k)
```

```
            count++;
```

```
    }
```

```
    path.pop_back();
```

```
int sum (Node* root, int k) {
```

```
    vector<int> path;
```

```
    int count = 0;
```

```
    solve (root, k, count, path);
```

```
    return count;
```

```
}
```

Q.4) Kth Ancestor of a node

Node Recursion.

I) Find (our node)

II) Backtrack (K → till K == 0).

```
Node* solve (Node* root, int &k,
             int node) {
```

```
    if (root->data == node) {
        return root;
    }
```

```
    Node* leftAns = solve (root->left,
                           k, node);
```

```
    Node* rightAns = solve (root->right,
                           k, node);
```

```
    if (leftAns != NULL && rightAns
        == NULL) {
```

```
        k--;
```

```
        if (k <= 0) {
```

```
            k = INT_MAX;
```

```
            return root;
```

```
        }
```

```
        return leftAns;
```

```
    }
```

```
    if (== && !=) {
```

```
    }
```

```
    return rightAns;
```

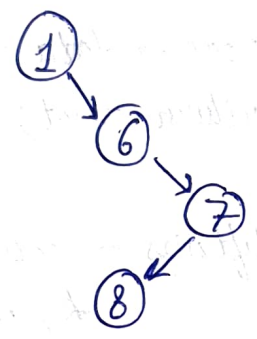
```
    return NULL;
```


lc 66

Q.1) Construct tree from preorder and ~~post~~_{in}-order traversal.

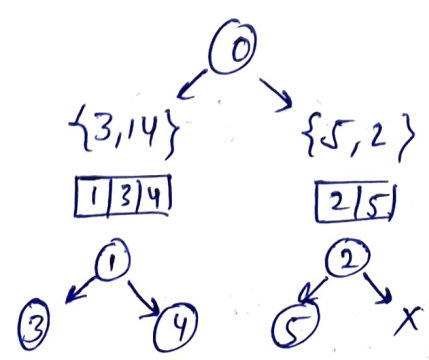
I/P → in-order[] = {1, 6, 8, 7}
pre-order[] = {1, 6, 7, 8}

O/P → 8 7 6 1
↳ post-order

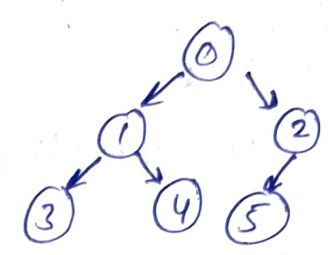


- Algo:
- 1) Take first element of preorder as root.
 - 2) Find pos of our root element in inorder.
 - 3) Uske left wala part → left subtree
right part → right subtree.

in = { 3, 1, 4, 0, 5, 2 }
pre = { 0, 1, 3, 4, 2, 5 }
 root



our tree :



code (using recursion)

```

Node* solve (int in[], int pre[],
int &index, int inorderStart,
int inorderEnd, int n,
map <int, int> &nodeToIndex) {
    if (index >= n || inorderStart >
        inorderEnd) {
        return NULL;
    }
    int element = pre[index++];
    Node* root = new Node(element);
    int position = nodeToIndex[element];
    root->left = solve(in, pre, index,
inorderStart, position-1, n, nodeToIndex);
    root->right = solve(in, pre, index,
position+1, inorderEnd, n, nodeToIndex);
    return root;
}

Node* BuildTree (int in[], int pre[],
int n) {
    int preOrderIndex = 0;
    map <int, int> nodeToIndex;
  
```

Node* ans = solve (in, pre, preOrderIndex, 0, n-1, n, nodeToIndex);

return ans;

}

lec 67

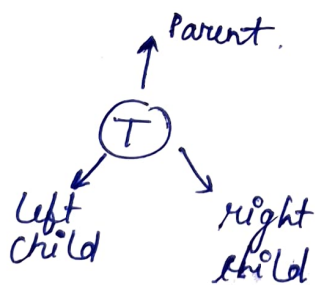
Q.) Minimum time to burn the tree from given target node.

approach

I) Make a nodeToParent mapping (level order traversal) to travel from bottom to up in a tree.

II) Find target Node.

III) make a map {Node*, bool visited} and keep adding nodes in queue from target Node (T).



IV) If (there is any new addition in queue)

→ true → time++

→ false → do nothing.

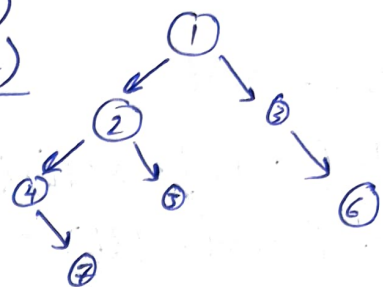
V) return time.

lec 68

Q.1) Morris Traversal of Tree

T: O(n)

S: O(1)



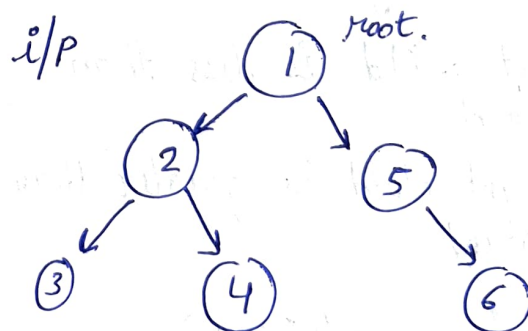
→ 4 7 2 5 1 3 6

→ it allows you to traverse the tree in-order (sometimes preorder) without using recursion or stack. (LNR)

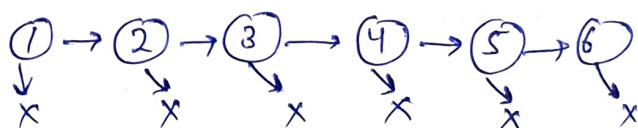
→ It modifies tree temporarily creating temporary links from a node's predecessor back to the node.

Q.) Flatten a Binary Tree.

→ make all left child NULL and right child next node of a linked list.



o/p



Morris traversal (Pre order)

N L R

Algo:

curr = root

while (curr != NULL) {

if (curr → left) {

· pred = curr → left

· while (pred → right) {

· { pred = pred → right;

· pred → right = curr → right

· curr → right = curr → left

· } curr → left = NULL;

curr = curr → right

}

}

~~// left part over~~

~~curr = root;~~

~~while (curr != NULL)~~